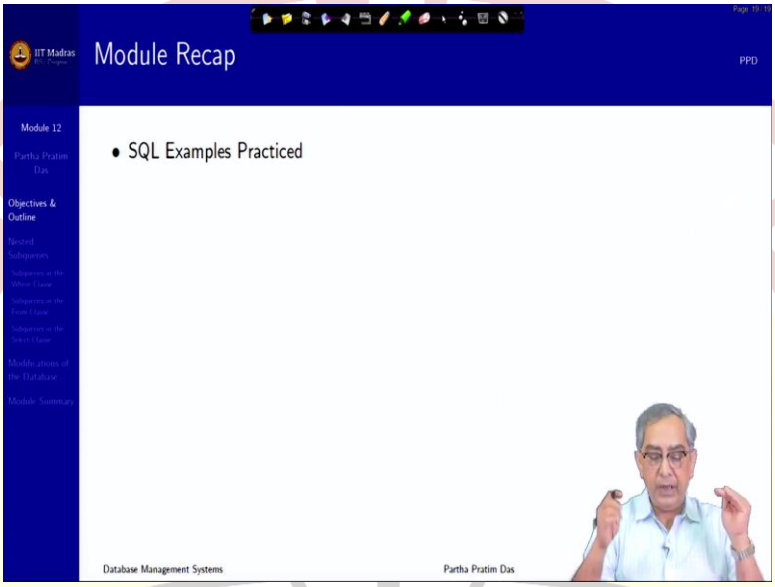
IIT Madras

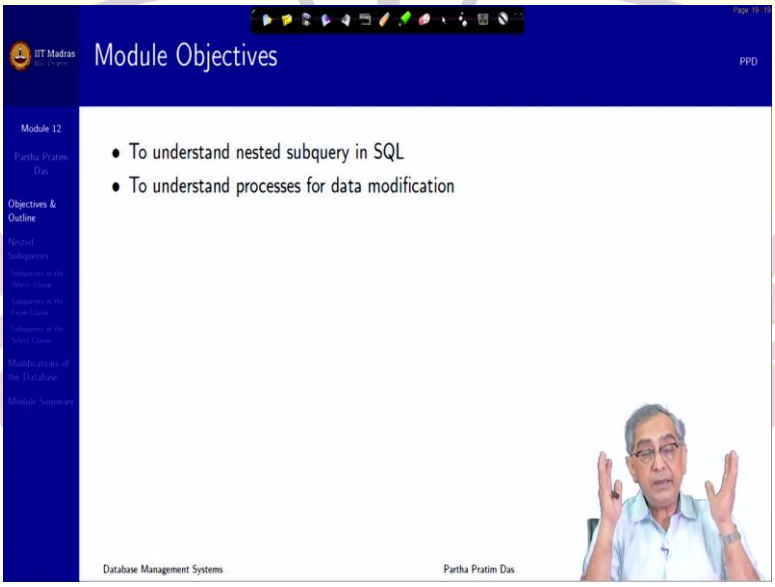
ONLINE DEGREE

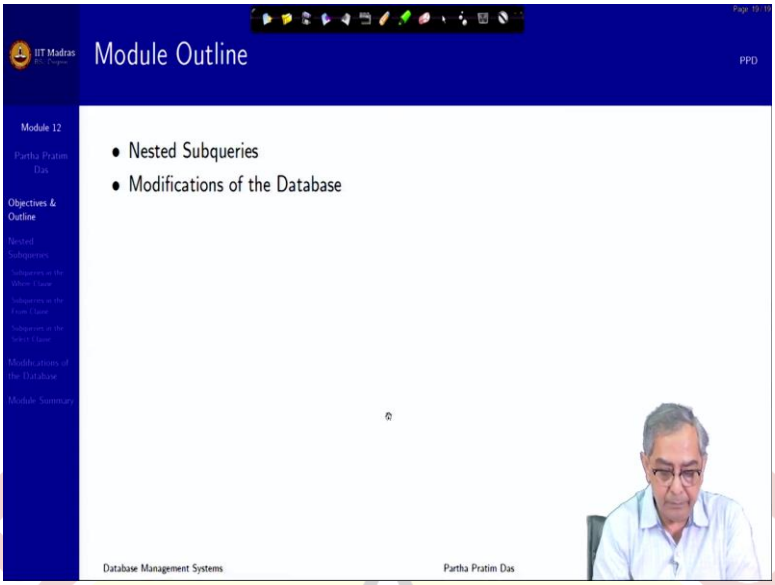
Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Intermediate SQL/1

Welcome to Module 12 of Database Management Systems course in IIT Madras online B.Sc programme.

(Refer Slide Time: 00:28)



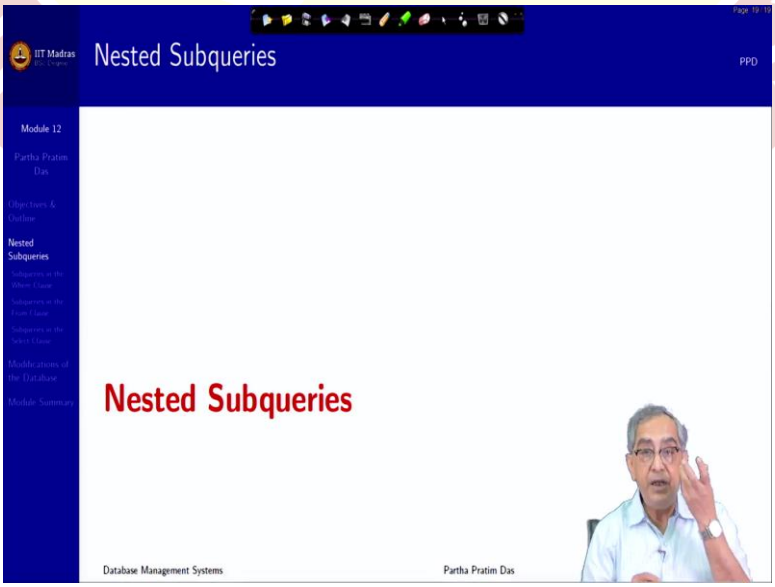


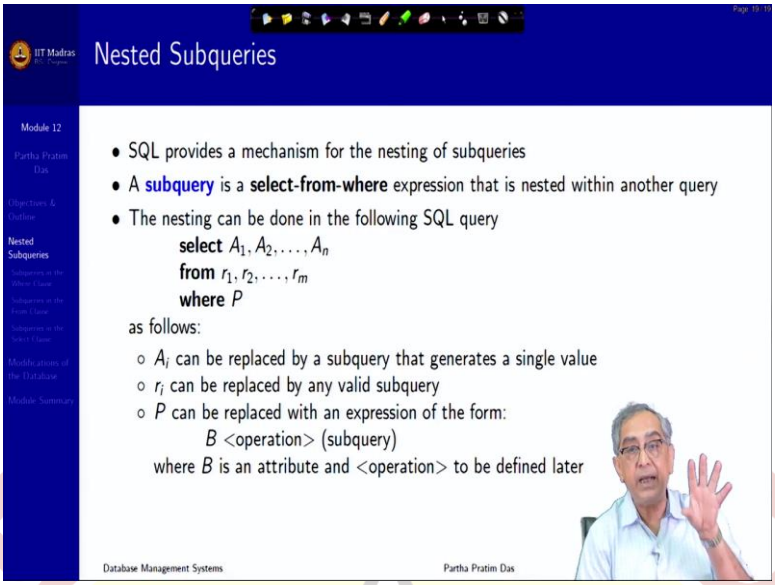


In the last module, we just did a kind of walk through with examples of the previous initial basic part or introductory part of the SQL. So, from this module what we start is what I am calling as intermediate SQL that is once the introductory or the basic query structure and how do they work, then you build up on this in the intermediate SQL to get, to be able to do really more useful stuff and finally, learn about advanced, certain advanced feature of SQL.

So, in the module today, we will talk about two basic and very powerful features. One is obviously absolutely necessary that is data manipulation. So, far, we are just talking about queries getting things out as they are, but how do we add records, how do we remove records, how do we change certain value of the attribute and so on is important, that is data modification. And the other is a powerful mechanism of nested sub queries.

(Refer Slide Time: 01:37)



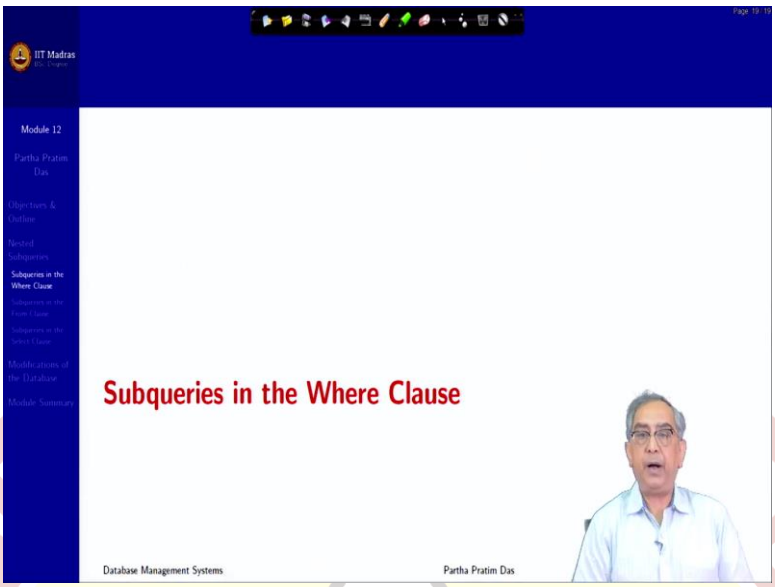


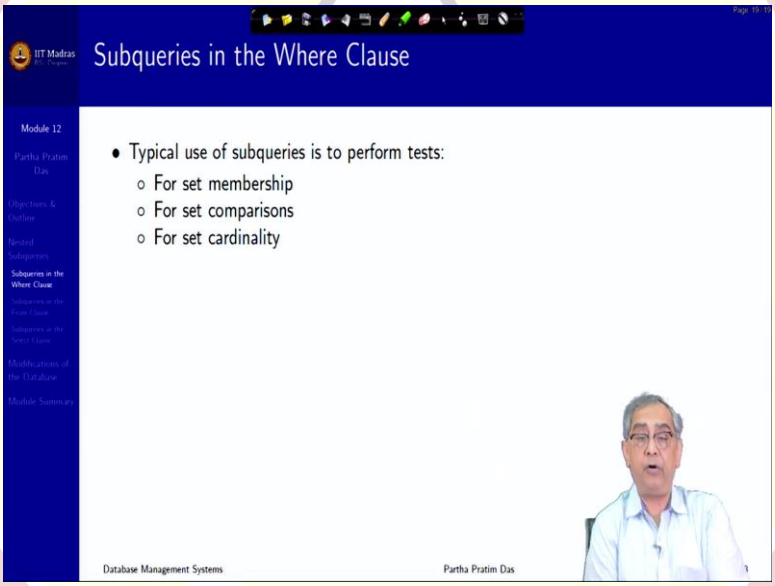
That is, these are these are features where one query can occur as a part of conditioning another query or a part of some other query. So, the SQL select from where structure that is the backbone expression structure within which we work, so, it has a set of attributes on select line, set of relations on the from clause and a total I mean maybe as complex as compound as it may be a predicate in the where clause.

Now, in any one of these, I can replace the particular item by a sub query, that is an attribute can be replaced by a query that generates a single value or the relation ri can be replaced by any valid query. So, what is the output of a query just reminding you the output of a query is input of a query is one or more relations, output of a query is a single relation always.

Now, so any query I have the output is a relation. So, I can use that output itself as a relation in the from clause to build up other queries. So, that is the nesting part, or I could be using it in the predicate condition, where B operation sub queries the form in which I write so where B is an attribute, operation is some kind of operator we will define later and a sub query. So, that is the basic structure.

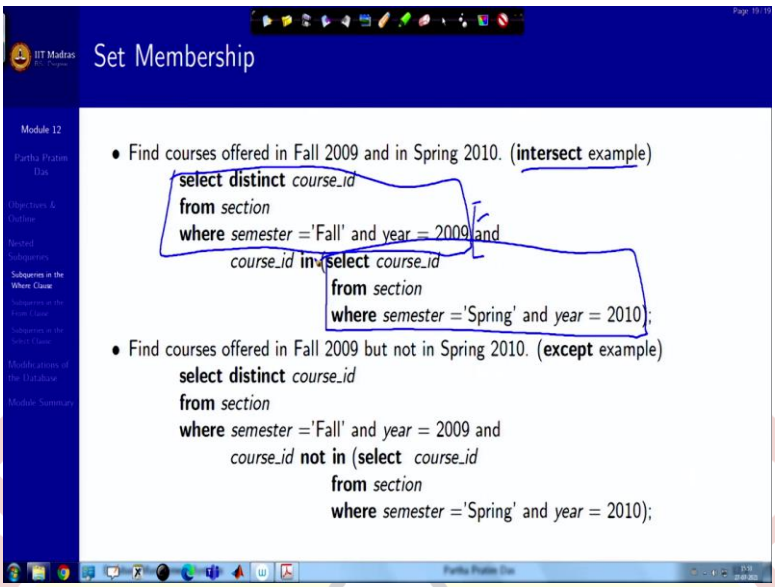
(Refer Slide Time: 03:17)





So, the sub query can happen as a part of where clause, as a part of from clause and as a part of select clause as we said, so we start with the where clause. Now, in where clause, a typical sub query is used to perform set theoretic tests that is, it can be used for set membership, comparisons or cardinality these are typical use, you could have other use also, but these are the typical use in a where clause.

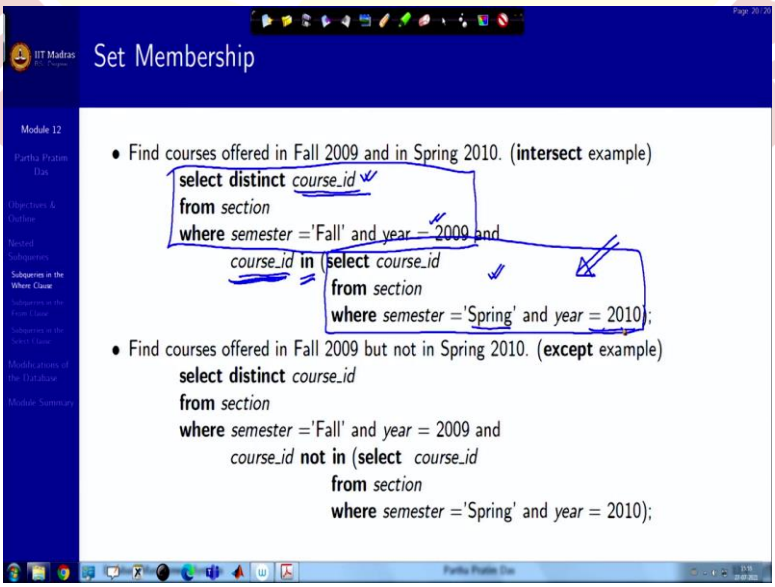
(Refer Slide Time: 03:47)



So, we are again going back to examples that we did in the last module that find courses offered in fall 2009 and spring 2010. We solve this using intersect, this a intersect example, now we do, to look at it something different, I look at it from the intersections or intersections fundamental definition point of view.

So, what is it the first part if you look at leave aside this and and subsequent. So, if you just look at this first part, what is this? This is the set of courses, which are offered in fall 2009. And what you will have to do, in intersect what did you do? We found out a set of courses offered in spring 2010 and did an intersect that is found out what is common. So, that commonality can be checked in a different way as in here, we say that this part is fine.

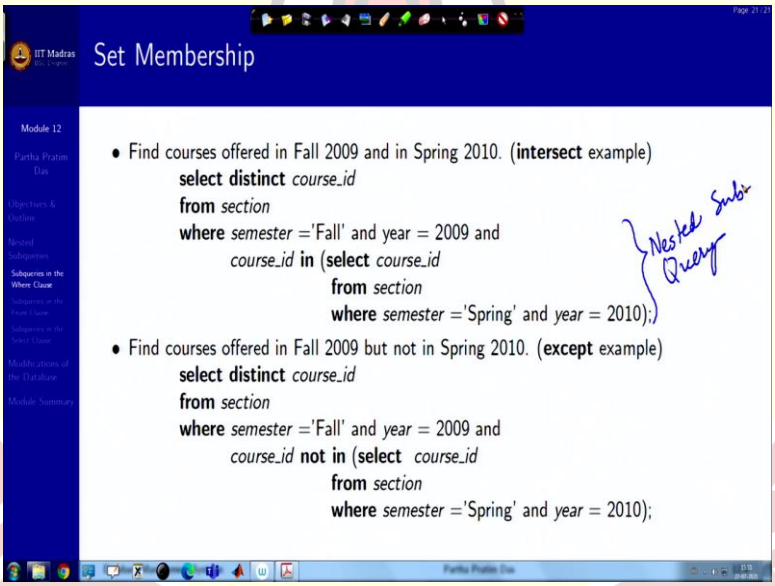
(Refer Slide Time: 05:04)



So, a tuple which will hump here is obviously a course that was offered in fall 2009. And then we say that same course ID, course ID that I am going to choose same course ID does it occur in this set, does it belong to the set that is the in, we have seen in as distinct values in fall, spring like that examples we have seen here it is a set, is a set membership after all, so in. So, this will also have a set of courses course ID's.

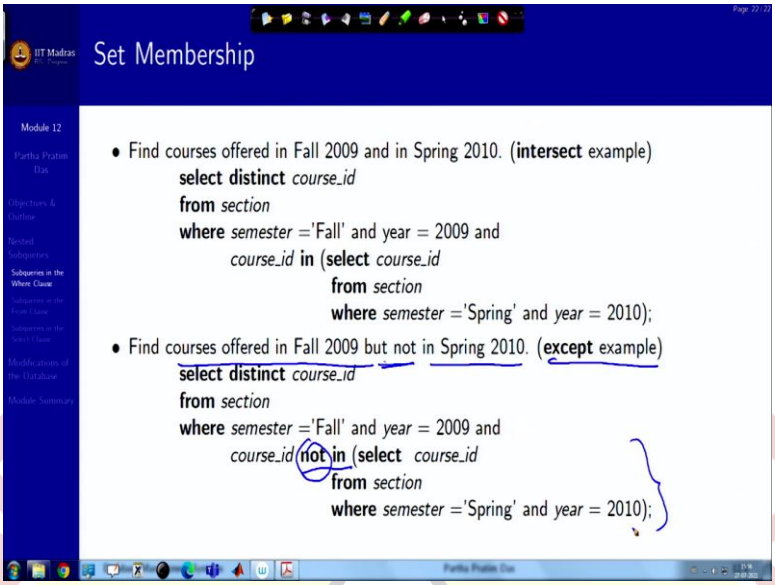
Now, if the course ID to be chosen here, which means it is offered in fall 2009 also belongs to this set that is it is in spring 2010 then obviously, it belongs to both and therefore, this is a part of the answer for this query. So, it is basically you can look at it from the intersection point of view, where you say that I take two sets and take the intersection or you can say that I take one set, take every element if that element belongs to the other set, then I choose in the result otherwise I throw it away, that is a same way intersection specified in terms of a predicate terms.

(Refer Slide Time: 06:32)



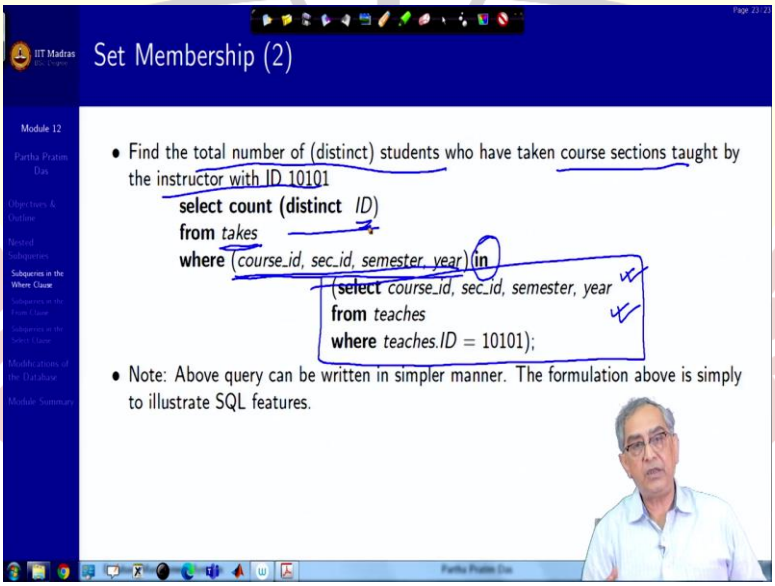
So, this is this part within bracket is a nested query and just to mean that nested query cannot come by itself. So, you often say is a nested sub query part of another query.

(Refer Slide Time: 06:52)



Similarly, courses offered in fall 2009, but not in earlier one was and in, not in which will, which is like the except example and which will simply come by putting a not here, because we are saying that it belongs to the courses in fall 2009 and does not belong to, so that is not in. So, this is another example of a nested sub the same example just used in a different way.

(Refer Slide Time: 07:21)



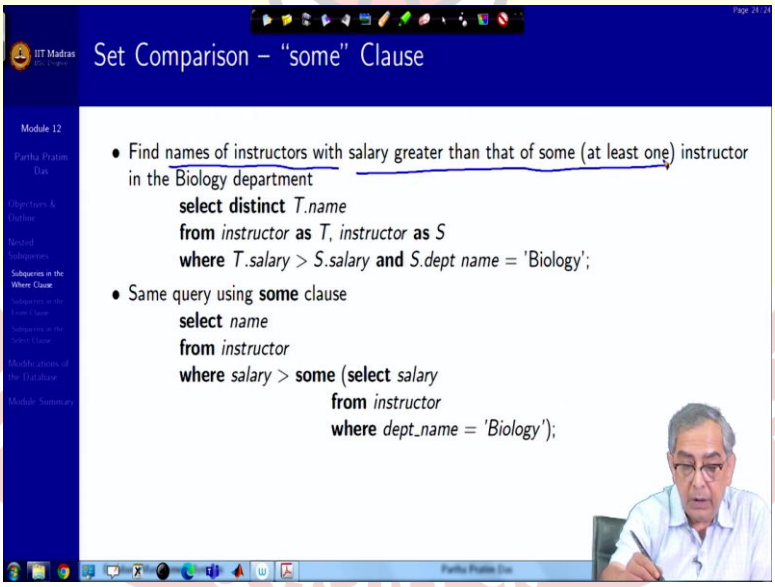
So, sub queries can be really very useful, you can do a set membership actually in a more, it will not have to be a single value for example we are saying, the total number of distinct students, total number of distinct students have taken course section taught by instructor ID this. So, how do we get there?

Now, the question is that I have done it in terms of a nested query, and that nested query tells me given the instructor ID, what are the courses that instructor has taught. So, a student may or may not have taken that course, if the student has taken that course, then that student will come in the count of the final result, and that student did not take the course that will not come.

So, these are the courses offered by the instructor. And this tuple course ID, section ID, semester year is from takes. So, which tells me given the student, given any student for that student, what is the course that student has taken.

Now, if this tuple belongs to this set or belongs to this relation, then the student has taken this course. So, the student should be counted. So, that is a set membership in which checks for it. And then I take the student ID, there could be multiple of that student. So, I want distinct so I made distinct on that ID and then I do a count, rest of it. Of course, I mean, this query is simpler, you can write it in a simpler form I know but I just wanted to show that you can do set membership not just with a single value but with the entire tuple altogether.

(Refer Slide Time: 09:34)

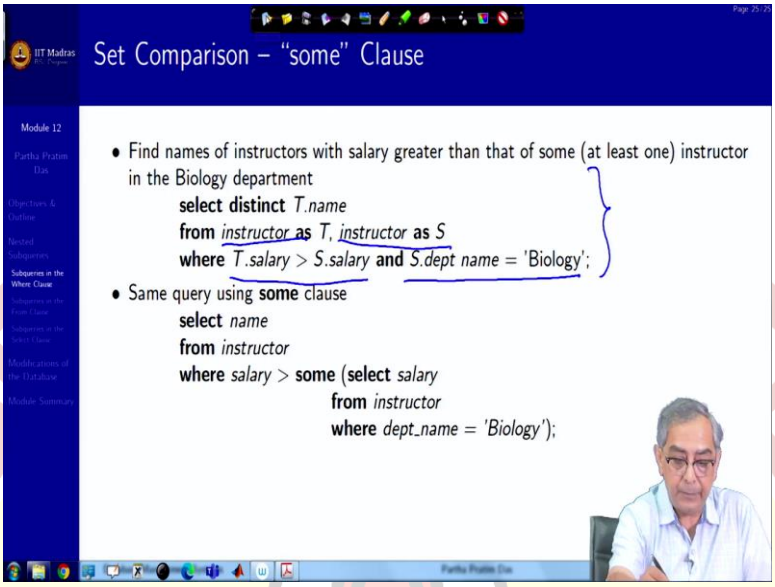


There is another more semantics on to this, which says that say let us look at what we want to find out. Names of the instructor, salary greater than some instructor in biology department, which means some that maybe the instructor's salary may be greater than everyone in the biology department to greater than five people in the biology department.

But there has to be at least one instructor in the biology department who must have a salary lower than the instructor we choose only then that instructor will come, if the instructor has a

salary which is lower than the lowest salary of an instructor in biology department, the name of that instructor will not count.

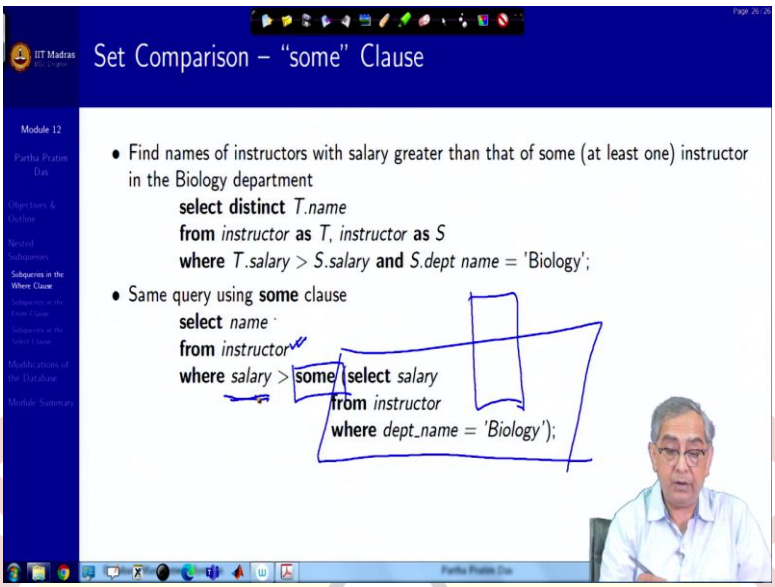
(Refer Slide Time: 10:28)



So, let us look at how we write this this is written in the way you know, so I have to choose instructor, I have again choose instructor because in the information of department as well as salary of the instructor both are in the instructor table, it is the case of a self-join. And I have to get a condition that T.salary is greater than S.salary.

It should be satisfied only when there is something, there is at least one tuple where the salary is less than the salary T. Given that the S.department is Biology, so we take the set of biology instructors and against that you check all instructors for their salary values. This is a simple way to do that.

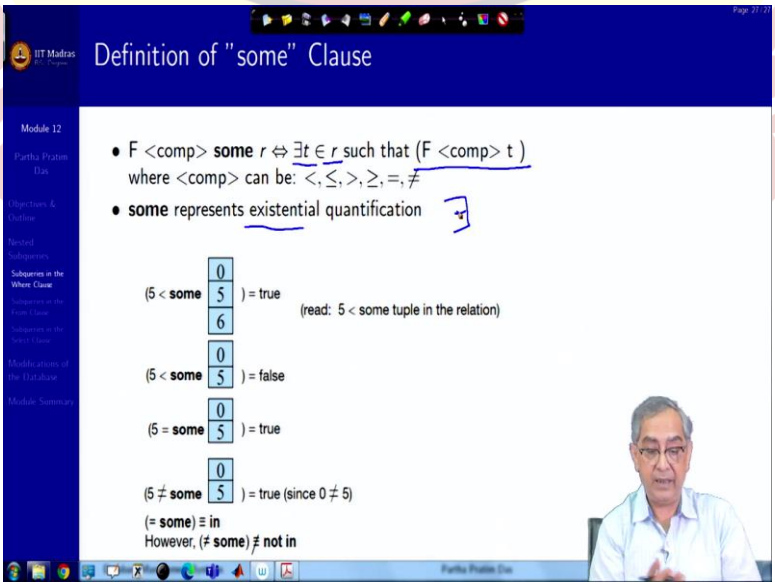
(Refer Slide Time: 11:28)



You can make it semantically clear if you say that well there is a sub query here, what is a sub query? Sub query is the set of salary of the instructors in Biology department you just need to check on that and from the instructor, you need to check if the salary is greater than at least one of them. So, that semantics is given by some so you say salary greater than some.

So, in the, in this part you have a list one column relation where there are salaries are given and here you are checking with the salary and seeing that this condition will be satisfied if the salary of one instructor is greater than at least one of them. If it is greater than none, that instructor's name does not come. So, this is.

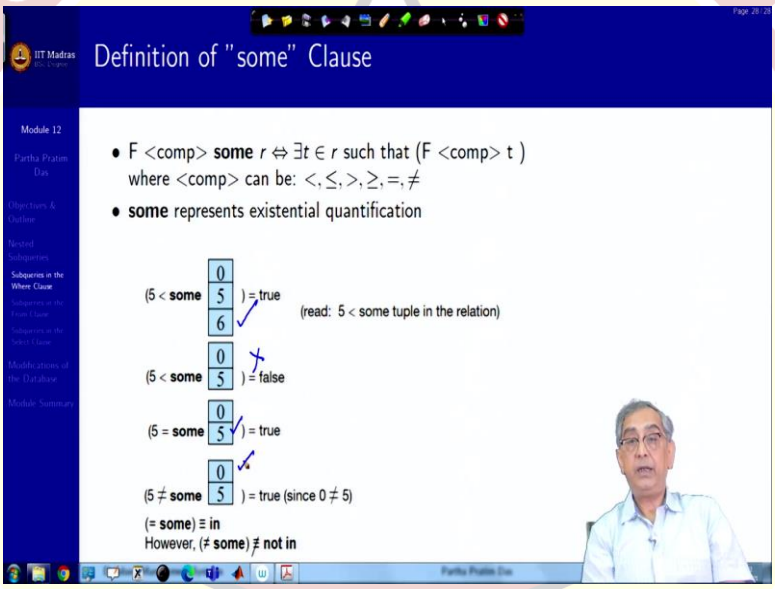
(Refer Slide Time: 12:26)



So, for those I told you to a little bit brush up on the predicate logic, for those who have done that, if you have not then please do it again. So, if I have f comp, that is comparison less than greater than any of the six comparisons some under relation, then what it necessarily means is this there exists a tuple, this means some means there exists a tuple t, and there is at least one tuple t in the relation. So, that this condition is satisfied.

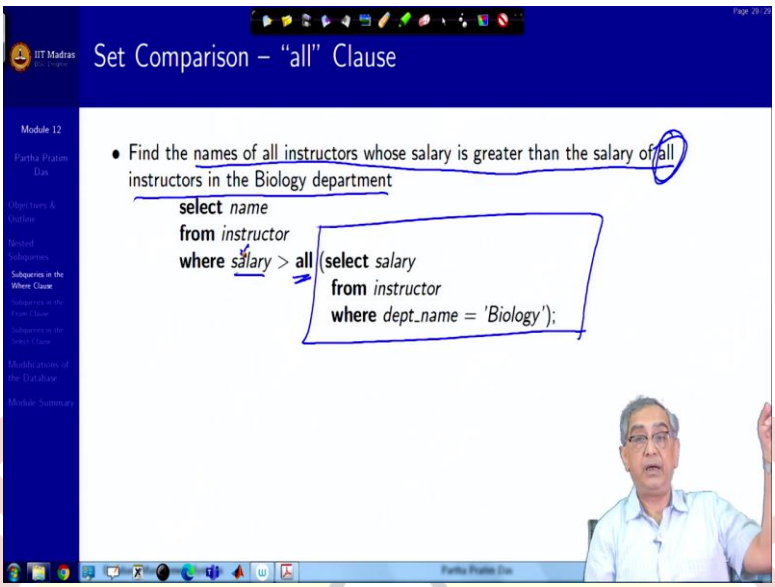
If there is no tuple t in r, whereas where F comparison t is satisfied, then this whole thing is false. But if there is at least one, then this whole predicate expression is true. And this is this there exists is known as existential quantification because it exist, so some basically represents an existential quantification.

(Refer Slide Time: 13:35)



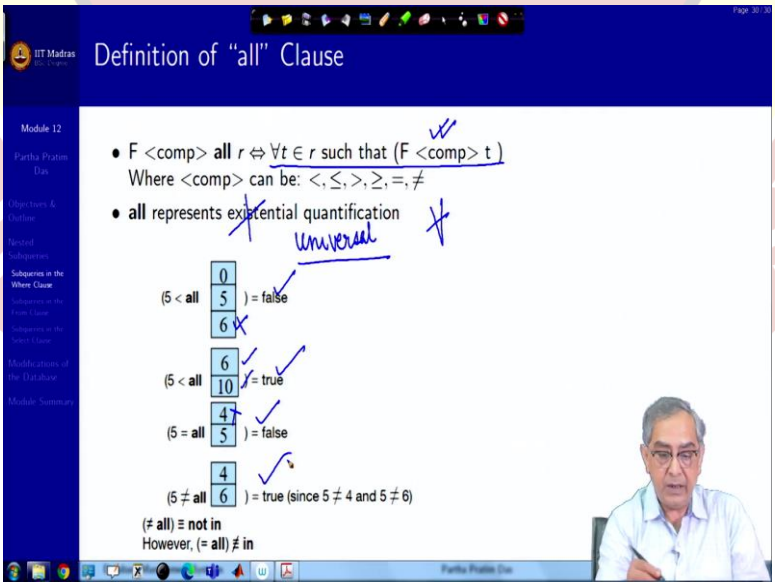
So, we can say five less than some, is it given this relation? Is it true? Yes, there is a 6. So, there exist 1, 5 less than 0 and 5, this is false, because 5 is not less than any one of them. 5 is equal to some. Yes, it is equal to 5. So, this is true, 5 is not equal to some. This is also true, because there is 0, so this is how. So, some is to be used for existential quantification in terms of the predicate logic in many cases.

(Refer Slide Time: 14:01)



Now, naturally if you talk about existential quantification, there will be individual quantification. So, that is what comes in the find the name of all instructors whose salary is greater than the salary of all instructors in Biology department. That is exactly the same thing as before, create the salary list, check for the salary, but you say all which means that each and every salary will have to be less than this salary for the instructor to be selected in the final result.

(Refer Slide Time: 14:40)

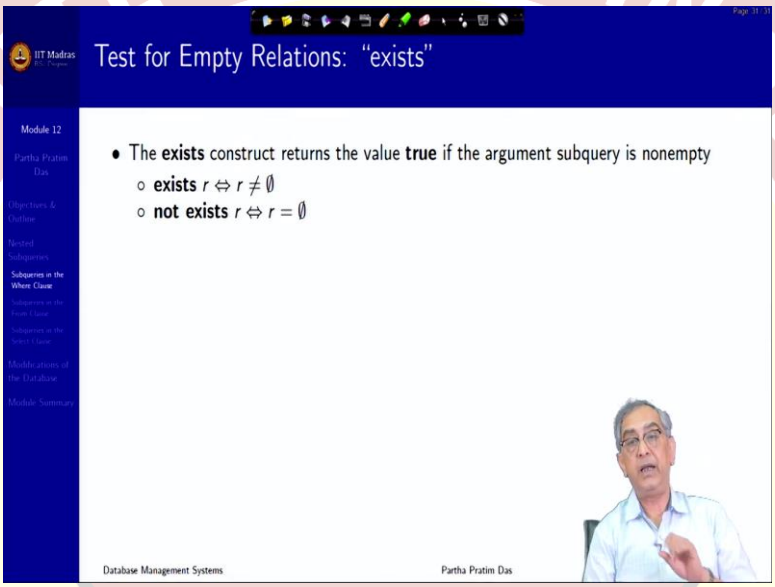


Which goes on to say that definition of a all clause is nothing but for all t. That is if I take each and every tuple of the relation r, then this condition must get satisfied if it does, then the whole condition is, whole predicate is true. I am sorry, this should be written as universal

quantification is not existential all represents the for all that is universal quantification, please correct that in your notes.

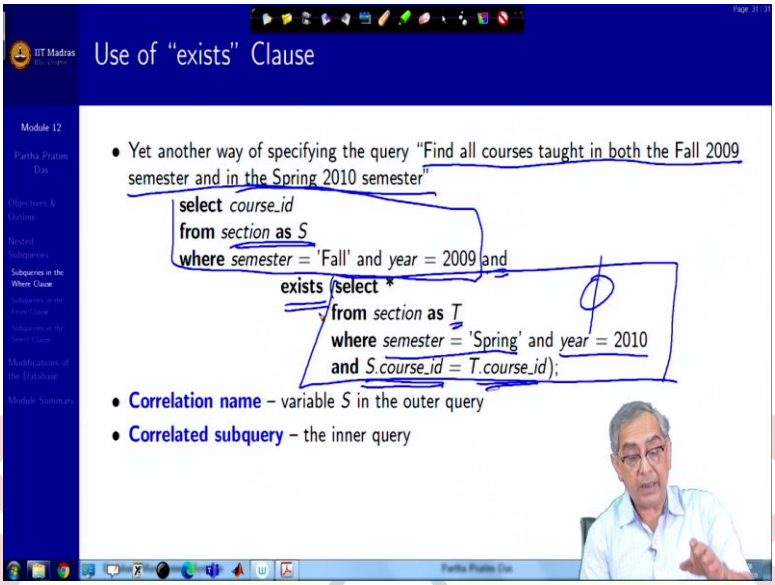
So, if we look at this example 5 less than all. No, it is not, it is not less than 6. So, it is false. 5 lesson all, true, less than both, so it is true, 5 equal to all, false, because it is not equal to here, 5 not equal to all, it is true, because it is not equal to 4, not equal to 6. So, this is how the glimpses of existential as well as universal quantification can be done in the in with the use of nested query and some and all qualifies.

(Refer Slide Time: 16:00)



Then there is another to check for empty relation called exists. So, exist is a construct that will return true if there exists at least one tuple in that. So, if you have done on r if you had done a count *, and if you get a value greater than 0, then exists is true. Otherwise, exists is false. So, not exist r is true, if there is no tuple, which means count * is 0 or the relation is empty.

(Refer Slide Time: 16:58)



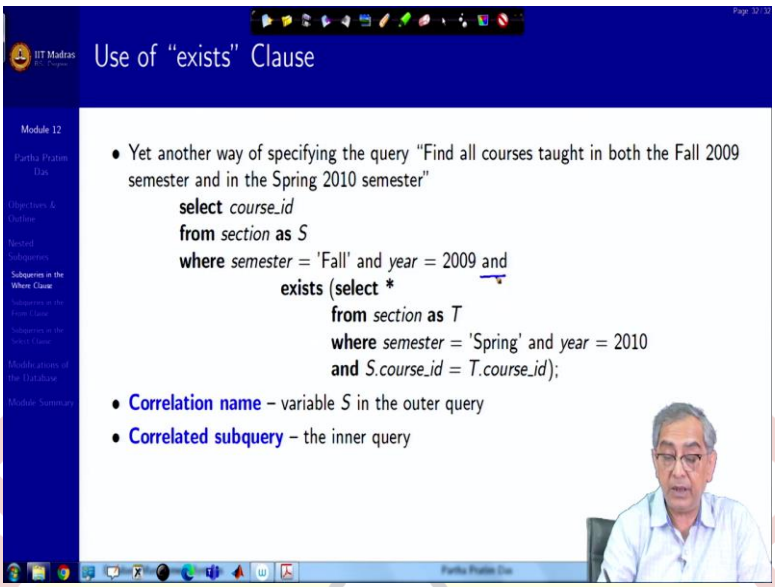
So, this can be used to also very effectively in different ways, smart ways. So, just check one, we are again trying to find all courses taught both in 2009 and 2010. The same query, we are just trying to show you that using different features, there are different ways you can semantically express it.

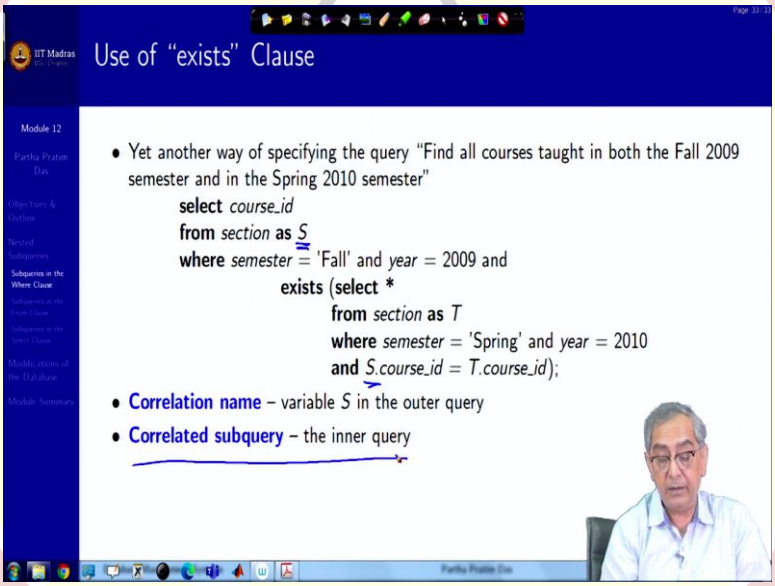
Now, in some cases is a matter of style, choice of the developer, choice of the programmer. In some cases, will see later, that certain ways of writing a query is more efficient than certain other ways of writing a query. And you will also have to keep in mind that, in some cases, unless there is a pressing need, we write queries in a way which are easy to read, easy to understand, because it is finally the human beings who have to debug and maintain the code.

So, we have to find the courses that exist in both. So, this part you understand, this part is courses in fall 2009. And so, I have to make sure that they are in spring 2010, as well. So, what we do is, I do the basic count kind of stuff, we said that semester is spring, year is 2010, because that is where I am looking for the course, I want to make sure that I look at the same course as here so that the same course is being offered, because back of your mind you know you have to do an intersection.

So, if I see that the same course is offered in spring 2010 then this sub query will be a non-empty relation, and the sub query will be non-empty relation there will be, there will exist something. But if this course S.course_ID was not offered in spring 2010 then there will be no tuple in T such that S.course_ID will match T.course_ID. And therefore, this in that case will become null. And that check is what we do by exists.

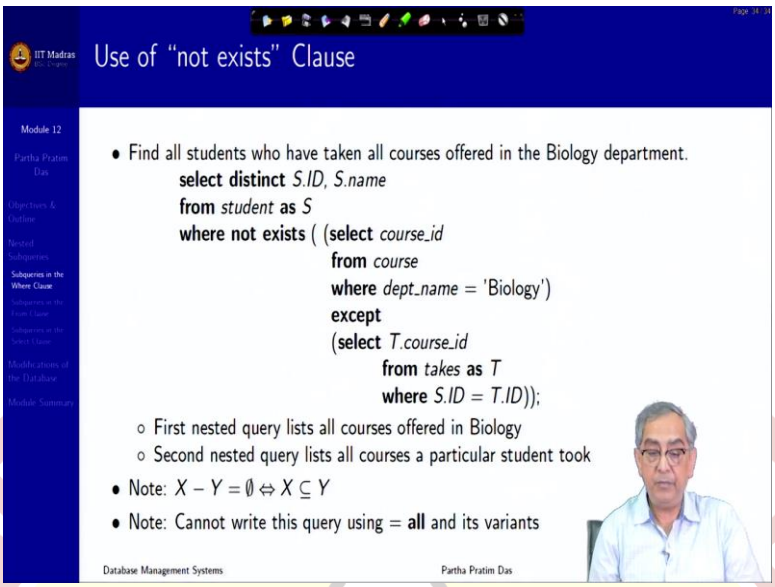
(Refer Slide Time: 19:18)





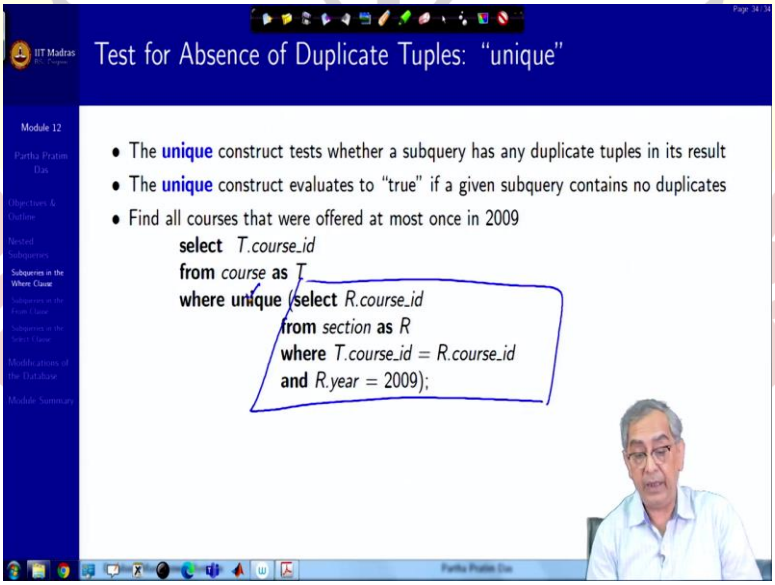
So, you say that take courses from relation S, which is courses in fall 2009. And check if that same course, if the courses that were offered in spring 2010. With that same course ID is whether that set is empty or not empty. If it is not empty that course is selected and condition ensures that. So that is a beautiful way of using exist. So, the correlation name here is S because you are correlating between the outer query and the inner query through this, and the correlated sub query is a inner query that we have got.

(Refer Slide Time: 20:00)



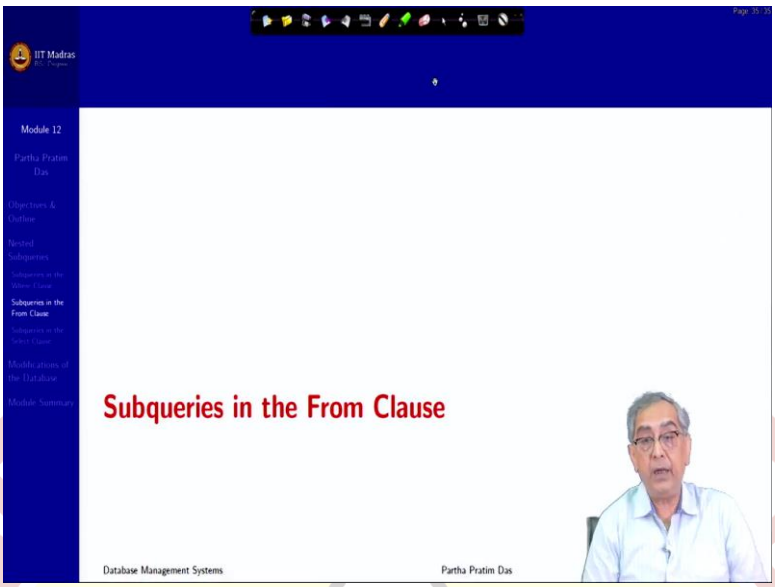
Now, as you can have exist, you can also have not exist. So, not exist is nothing but saying that it is empty, the set is have to be empty, I will not go through the explanation of each and every you can see that here we have a sub query, which by itself is a difference of two queries, and then we are using not exist on that and I will request you to go through this, try to understand this on your own. And if you have problems, then you get back to us.

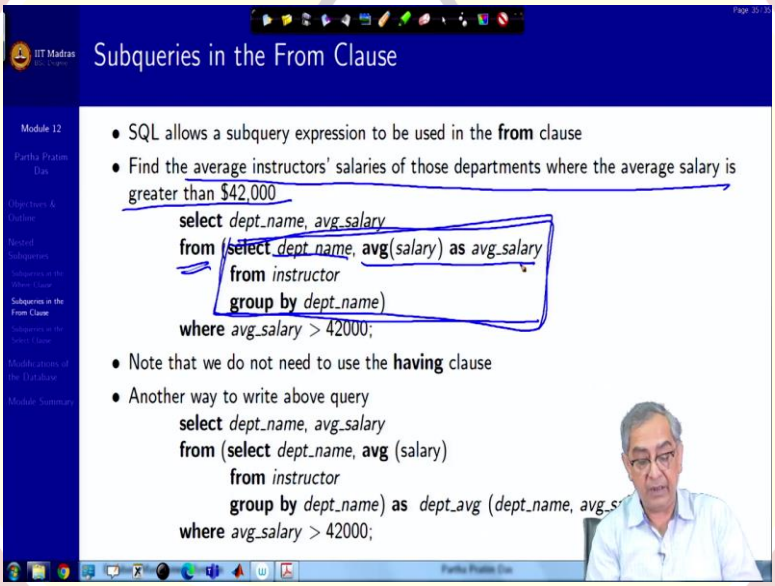
(Refer Slide Time: 20:40)



There is a way to check for uniqueness of values also. So, in a sub query like this, as you have here, if you have say unique, then that will be true provided all tuples returned at unique, otherwise it is false.

(Refer Slide Time: 21:02)



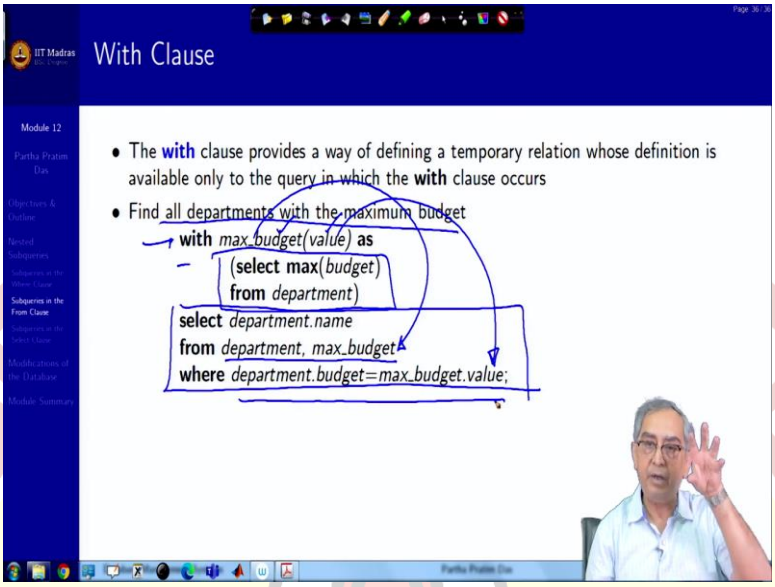


Now, this was in the where clause, but you can have sub query in the from clause which is actually easier. So, you are trying to find the average salary of those departments whose average salary is greater than 42,000. So, here is a sub query, which gives you the average salary of departments. And then you just do a select from where, simple.

You can mix the conditions and do it as well as you have seen. But here, so this entire thing is a relation, so you are using it as a from. So, you are saying that, what am I choosing from? I am choosing from the relation which has department name, and average salary, just take out that part from the instructor, make that abstract relation make that new temporary relation. And from that relation, you try to see the average query, and you do not need to use the

having clause. Otherwise, you could have, we have solved it earlier using having clause, there is another way of solving this, again, which we should check out yourself.

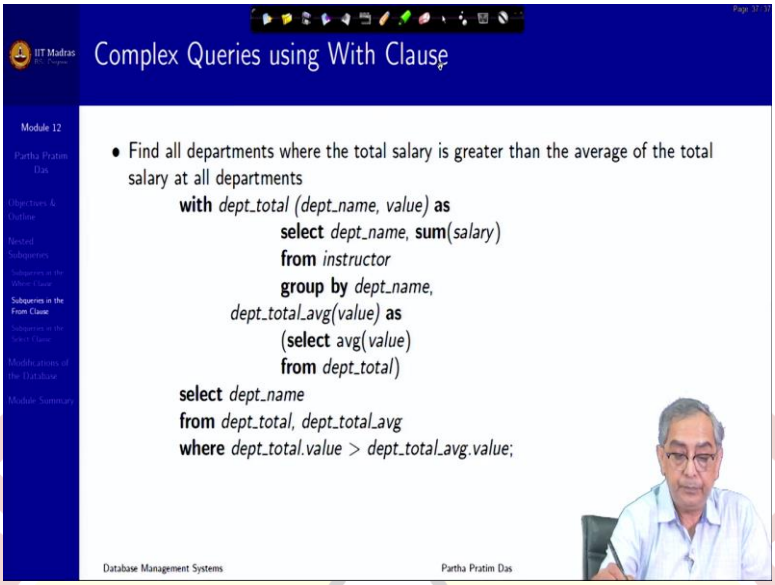
(Refer Slide Time: 22:12)



There is a with clause also, with clause, the semantic is simple. So, this is what your basic query is. So, you are saying that select department name from department and maximum budget, find all departments with maximum budget. So, with all departments which has the with there, which has the maximum budget, so what I do here, here we have a sub query from the department find the maximum budget it is a value.

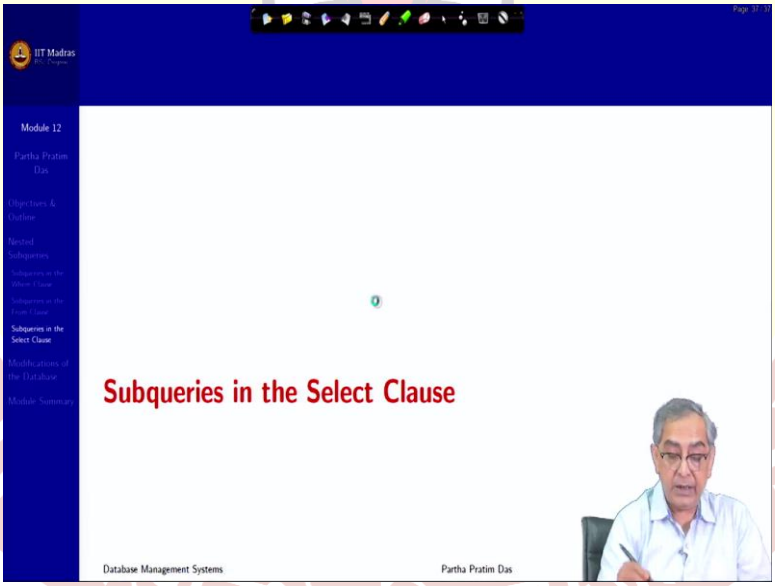
Then with that becomes a table called max budge. And the attribute is value. That is what you are specifying here. Now, I use this table here. And I use the value here to check that the department's budget is equal to the max budget. So, by with what you are actually doing is you are creating a temporary relation, which you can use in your query. So, that is the reason that this sub query actually kind of is a part of your from, but occurs before that, because first you will have to create that temporary before you can actually use it.

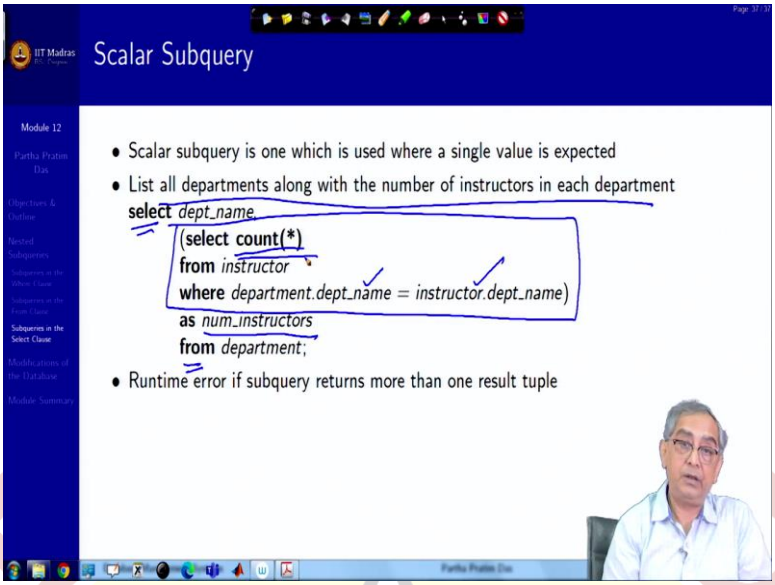
(Refer Slide Time: 23:48)



There are more complex examples with of queries using the with clause which I will leave to you for understanding at home.

(Refer Slide Time: 23:58)

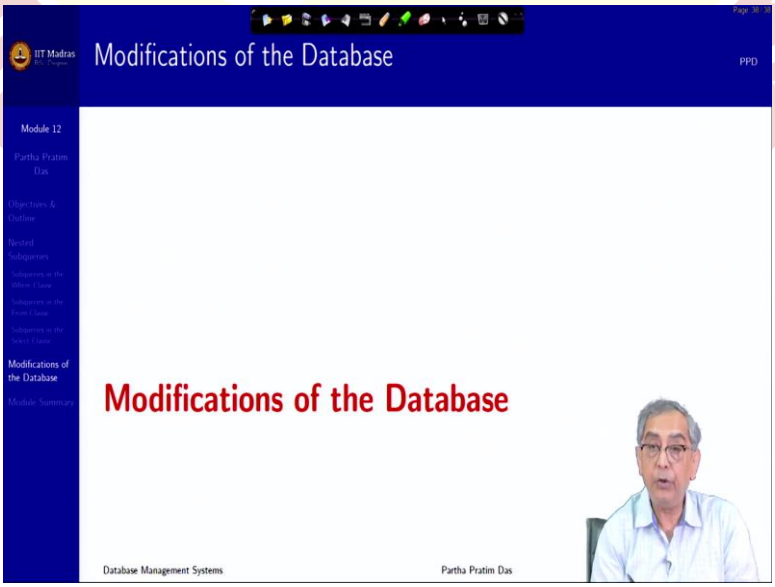


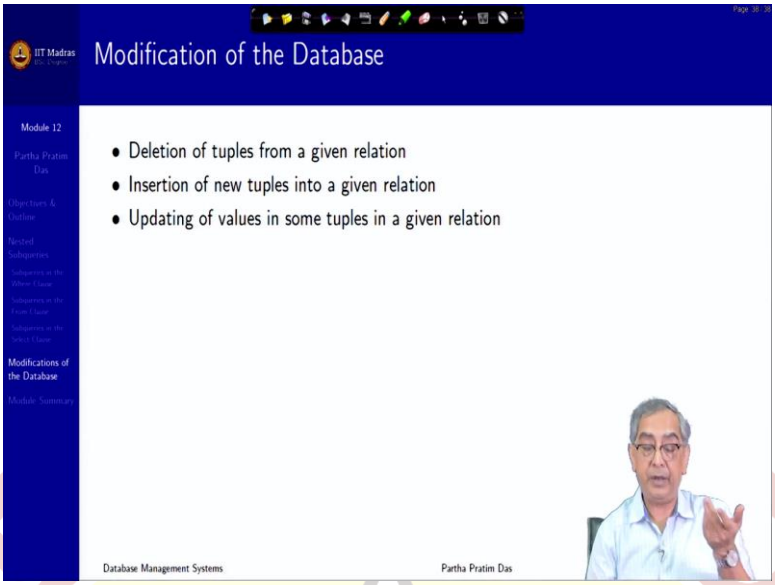


You can have sub queries in the select, but remember that select is always an attribute. So, it will need a scalar query, a scalar queries is where a single value is expected. So, when you say that list of all departments along with number of instructors in each department, so I have here a sub query and you can see it is a part of select because from comes later on.

So, what will this give me? This will give me a value. If I equate these two-department name from department and department name from instructor when these two are equated, then I will get that how many are there? How many instructors are there in that department? So, it is a single value. So, I call that as my field number of instructors. So, this is important that the sub query has to have a single one tuple result. If it does not, then this kind of a sub query will generate an error.

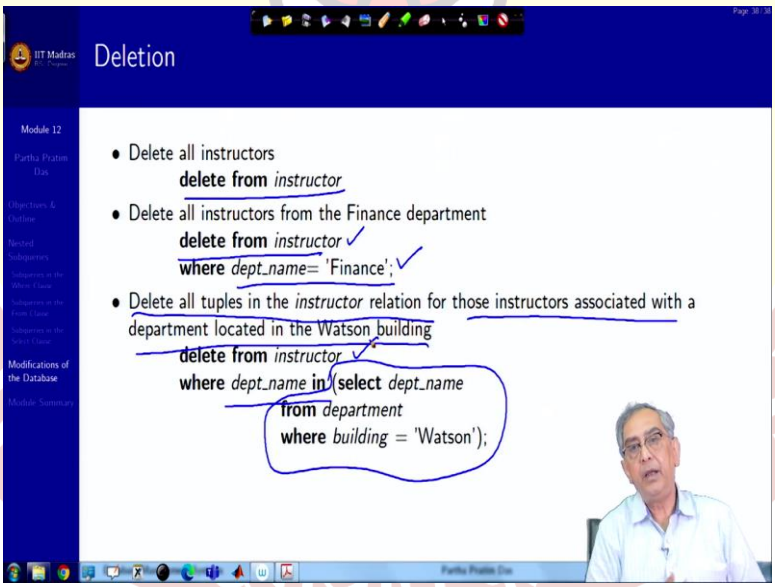
(Refer Slide Time: 25:17)





Having done that, let us quickly talk about how do you modify databases obviously, there are three things you can do, can delete a tuple, you can insert a tuple or you can update a tuple, these are very simple very similar to what you have already done in terms of select from where but the syntax is still different.

(Refer Slide Time: 25:30)

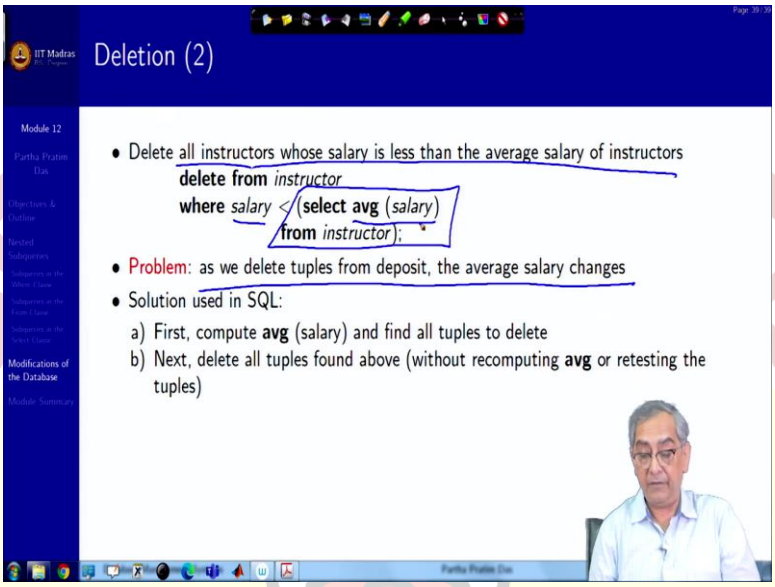


So, delete if you want to delete all tuples delete from the table that is it, others you can delete and also have a clause, where clause predicate. So, only those tuples which satisfy. So, what will this do? This will delete all instructors for the finance department. So, anything that satisfies this any tuple will be deleted.

Delete all tuples in the instructor relation for those instructors with the department located in the Watson building. So, kind of you are using a nested query here, so we find out the

departments in the Watson building. So, if the department is in the Watson building, then you delete that instructor written in very simple terms. So, kind of when you to delete, the conditions can be done very similarly, as you are doing any other where clause condition they could be simple predicates, they could be nested query predicates and so on.

(Refer Slide Time: 26:47)

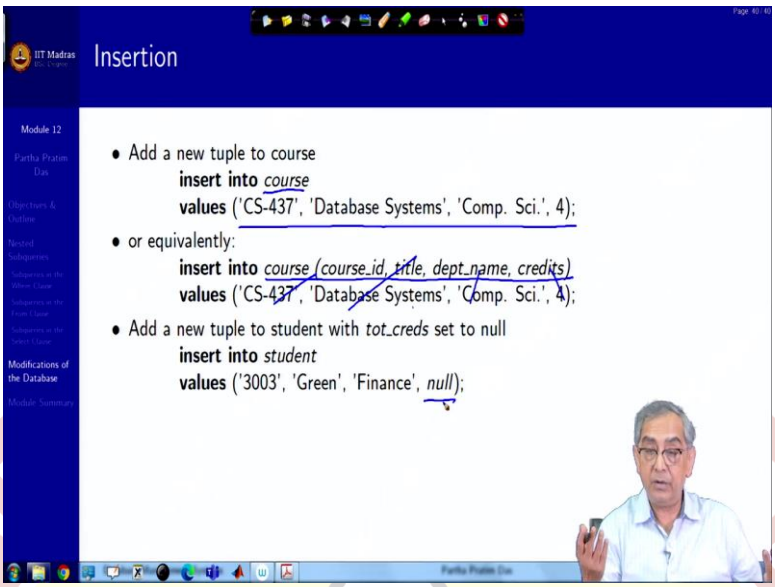


Look at this carefully, delete all instructors whose salary is less than the average salary of instructor, so we will say okay that is very easy, because I will do a nested query, average salary of instructors and say that salary is less than that, but there is a problem. The problem is you are now modifying. So, the moment one such instructor is found and deleted, the average salary of instructors themselves has changed.

Suppose there were 10 instructors, they had average, now one qualifies, because r salary is less than the average salary. So, you delete, once you have deleted your 9 tuples. So, the average is a different value. So, as we delete from the deposit, average salary changes, so this query will not work as expected.

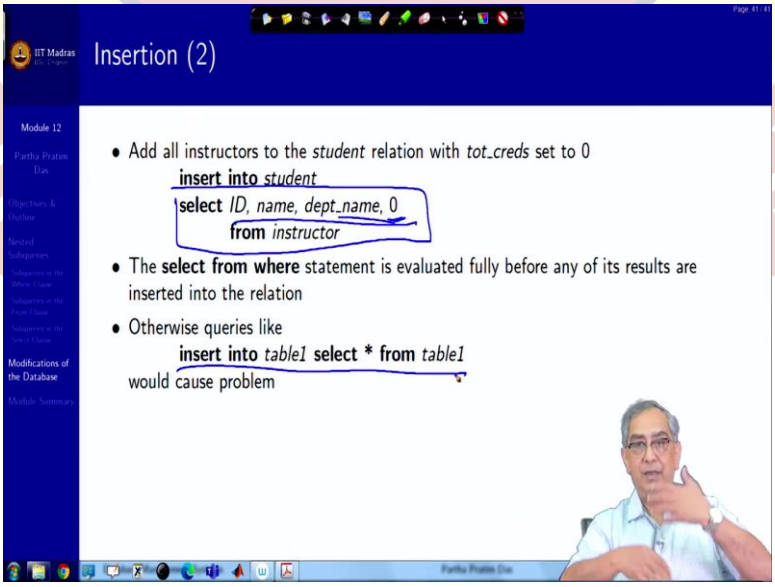
So, what we will have to do? Separate these two out, otherwise you are changing to that same table and you are using a condition from that change values. Instead, you have to first find the average in using some nested query or some mechanism you have to put it in a value and then delete all tuples which has a value which is less than that average value. So, these are things that you have to be careful about.

(Refer Slide Time: 28:17)



Insert you can do in a similar way, but the syntax is insert into, you say insert into the relation and then by values you give a tuple. So, this should be in the order in which the attributes have been defined in the create table, otherwise, it will be an error. So, you can either say specify just the course then you have to remember the order or you can get the course along with the name of the attributes in that case, the value should be given in the order in which you have specified the attributes very easy. You can also insert with null values just say null, total credits initial is not known for a student. So, you can just say null.

(Refer Slide Time: 29:13)

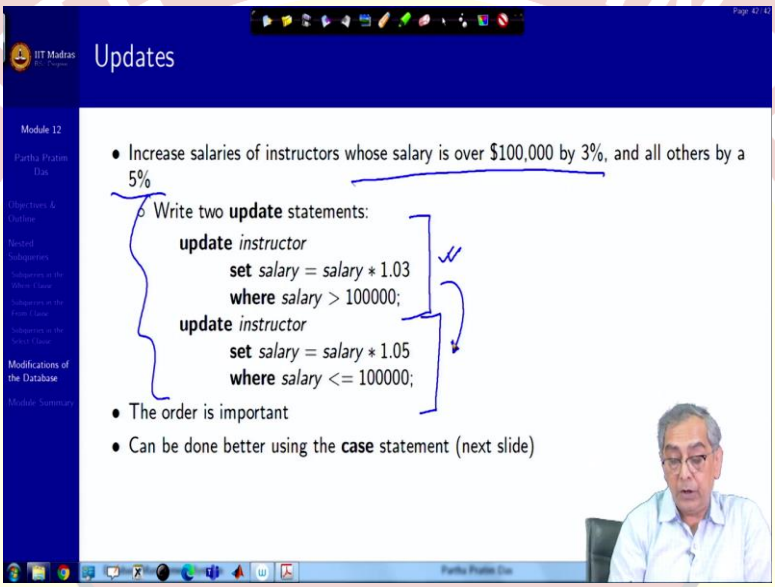


Now, when you do that, you can also set it to 0 as we have done in this first case, that is select from select ID name department, these are what you have got from the instructors table

and inserting into the student table for all those students, but the total credit is not known, so we are using the value 0.

So, in this way you can have you can have any such conditions. Again, there is a question of changing the table as you do this insertion. So, what it does is select from where this part of the clause is fully evaluated before the actual insertions happen. So, otherwise this will have a problem insert into table select * from table you will have a problem because it will keep on inserting.

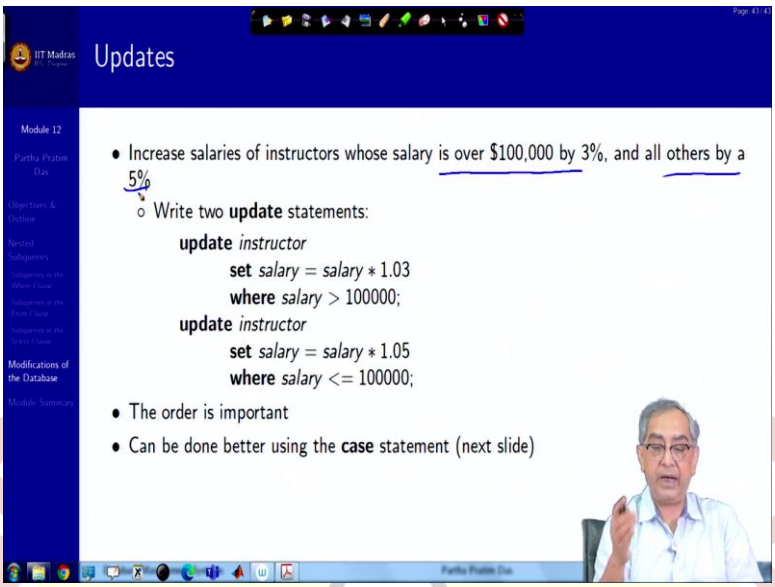
(Refer Slide Time: 30:22)



So, finally updates you can make changes to a field, you say update and the relation and then you through set you set the new value for the relation. So, here is an update to which updates what this does? It gives 3 percent raise to those who have salary more than 100,000, this another will get 5 percent raise if their salary is less than equal to 100,000.

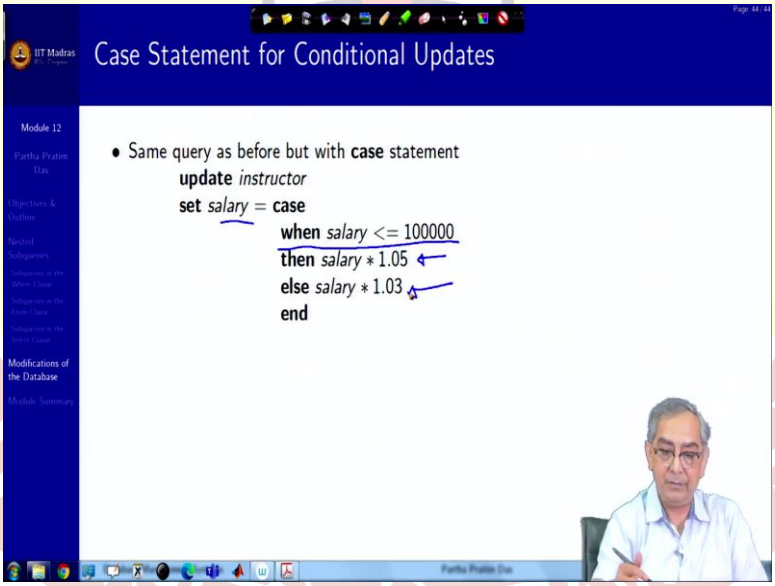
Now, the problem is if you want to do both, that is if you want to give raise of 3 percent for salary more than 100,000 and others by 5 percent, you will, you may have corner case conditions because by this update, you have already changed the values when you are using it here. So, it will be good because, so, it will be good to actually use.

(Refer Slide Time: 31:32)



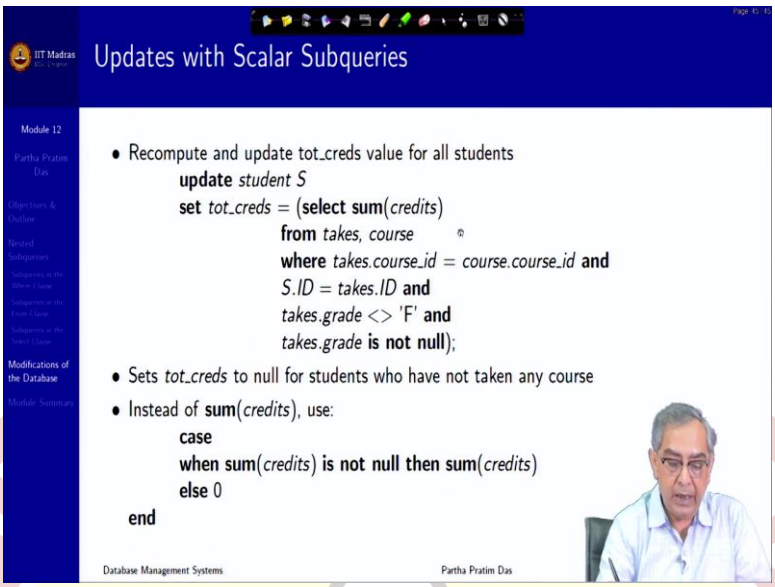
So, you can say that this here it is very clear that it is if this then do this else do this. So, very traditional if else kind of notion that we have in our procedural languages coming in.

(Refer Slide Time: 31:41)



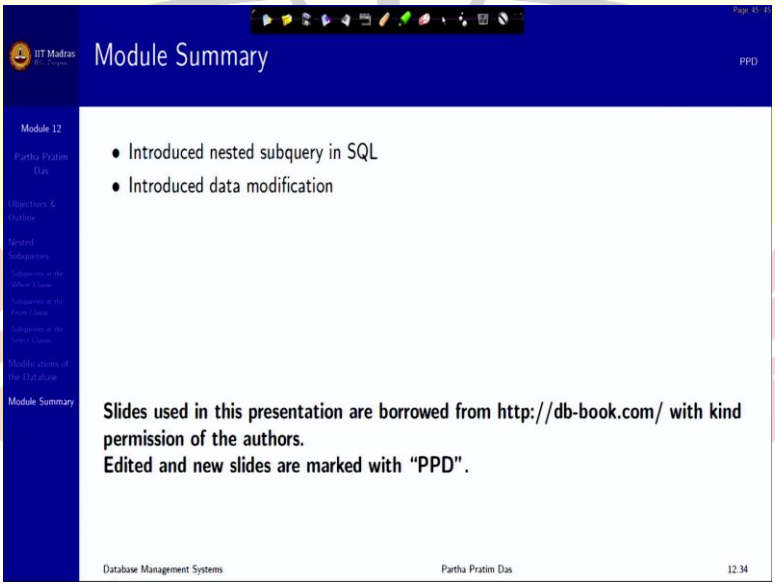
And something very similar exists here where you say you have something called a case statement, say salary is the new salary is case when there is a condition if it is true, then it is this, if it is false then it is this, will happen in one go, no problems.

(Refer Slide Time: 32:14)



So, these are all different features which make it easier for you to write queries. Then you can do updates with scalar queries also, what is the scalar query remember which returns only one result. So, you are here you are trying to do one of that, again I will not go through the specific steps you should be able to figure out yourself what is going on and ask otherwise.

(Refer Slide Time: 32:38)



So, in the module today, we have introduced nested sub query in SQL and we have also talked about the data modification features. Thank you all very much for your attention. And we will meet again for the next module on SQL.